Homework 5: Light Introduction to Trees

CS 150 with Dr. Sam Schwartz

Upload to Canvas when done.

1 Purpose

In this assignment you will be given a very brief introduction to trees. While you will learn much more about trees and other data structures in future classes (particularly your *Data Structures and Algorithms* class), we introduce the concept of a binary search tree here in CS 150 to also strengthen our command of objects interacting with other objects.

2 Tasks

Task 1

Create a binary search tree class called `class BST`, which is composed of nodes containing strings. These nodes are to be sorted by the default alphanumeric sorting order of Java strings. (That is "a" comes before "b"). As part of this, write a class called `class BSTNode` which takes a string in the constructor and saves it to an instance variable.

Task 2

In your BST class, write a method `public void insert(String s)` which inserts the string argument `s` into a BSTNode, and then the node into the tree.

Task 3

In your BST class, write a method `public int height()` which calculates the height of the tree. Do this recursively by utilizing a recursive helper function with the signature `private int height(BSTNode root)`.

Task 4

In your BST class, write a method `public void printDepthFirst()` which prints the contents of the tree in a depth-first manner. The output should be all on one line, with spaces separating the strings.

Do this with a recursive helper function.

Task 5

In your BST class, write a method `public void printBreadthFirst()` which prints the contents of the tree in a breadth-first manner. The output should be all on one line, with spaces separating the strings.

Use a `LinkedList` to help you accomplish this.

Task 6

Copy-paste verbatim this main method, adjusting for imports so that it runs.

```
public static void main(String[] args){
 System.out.println("Tasks 1, 2, and 6 are running");
 BST tree = new BST();
 tree.insert("U");
 tree.insert("W");
 tree.insert("E");
 tree.insert("A");
 tree.insert("U");
 tree.insert("C");
 tree.insert("L");
 tree.insert("A");
 tree.insert("I");
 tree.insert("R");
 tree.insert("E");

 System.out.println("==========");
 System.out.println("Task 3 output:");
 int height = tree.height();
 System.out.print("Tree Height: ");
 System.out.println(height);

 System.out.println("==========");
 System.out.println("Task 4 output:");
 tree.printDepthFirst();

 System.out.println("==========");
 System.out.println("Task 5 output:");
 tree.printBreadthFirst();
}
```

Task 7

Take a screenshot (or multiple screenshots) of your terminal showing the full output of `main()`. Upload this screenshot(s) along with a .zip folder of your code to Canvas.

3 Grading Criteria

In general I am looking for the elements of validity, readability, and fluency in all code-based assignments. (See more below.)

I tend to dock 5ish points off for each error (although smaller or larger quantities like -1pt or -10pts exist based on the magnitude of the error), and will provide free-form feedback detailing why any points were missed in the comments on Canvas.

Please note that I do not get notifications about replies to my comments on Canvas, so if you have any questions please reach out to me directly.

Validity

Student submitted files which implemented the tasks correctly and handled possible errors gracefully.

Moreover, the student submitted screenshot(s) displaying the code's output.

Readability

The deliverable used professional English and typesetting throughout, and had all methods annotated with JavaDoc.

Fluency

The deliverable was executed in such a way that an experienced practitioner would not find the deliverable "weird-in-a-bad-way" or unduly jarring.